

Alte. Guillermo Brown



E.T. N°36 - D.E.15

Desarrollo de Sistemas

FitPocket: Documentación

Profesor/a: Enzo Stella

Curso: 6°1°

Integrantes: Thomas Barrera Fuentes – Mauro Beltran – Lautaro Palombo –
Sofia Power

Versión actualizada: octubre de 2026

Índice

Sobre esta versión	4
Descripción y Alcance del Proyecto.....	4
¿Qué es FitPocket?	4
La experiencia diaria con FitPocket	4
Comunidad y progreso compartido	5
El equipo detrás de FitPocket	5
Descripción de Usuarios de Sistema	5
Usuario Invitado	5
Usuario Registrado.....	6
Usuario Administrador	6
Funcionalidades y Tecnicismos.....	6
Información del Usuario	6
Plan de entrenamiento y dieta	8
Registro de Avance.....	8
Vista de su Perfil.....	8
Comunidad	9
Compartir Avances Propios	9
Ideas para Comunidad	9
Idiomas	9
Requisitos Funcionales y No Funcionales	9
Funcionales	9
Requisitos no funcionales	11
Tecnologías y Herramientas.....	12
Frontend	12
Backend.....	12
Base de datos.....	12
Inteligencia Artificial y servicios externos	12
Seguridad	13
Despliegue.....	13
Diagramas.....	13
Diagrama de Casos de Uso	13
Diagrama de Clase	15

Escalabilidad 16

 Infraestructura y base de datos..... 16

 Funcionalidades de Usuario..... 17

 Integraciones Externas 17

 Seguridad y Control 17

Sobre esta versión

Este documento actualiza la documentación original de FitPocket para que describa el proyecto tal como está hoy (rama development, 8 de octubre de 2026). Cada afirmación sobre el estado actual fue contrastada con el código.

Criterios que se siguieron:

- No se eliminó ningún requisito ni funcionalidad del documento original. Los que todavía no están desarrollados se mantienen y se marcan como **Pendiente**.
- Donde lo construido difiere de lo planificado, se corrigió la descripción (por ejemplo, el onboarding pasó de dos a cuatro pasos).
- Se agregaron los requisitos y funcionalidades que se implementaron y no figuraban en la versión anterior.
- En las tablas de requisitos y casos de uso se agregó una columna **Estado**: Implementado, Parcial o Pendiente.

Descripción y Alcance del Proyecto

¿Qué es FitPocket?

FitPocket es una solución moderna e inteligente pensada para todas aquellas personas que saben que necesitan modificar sus hábitos para alcanzar un mejor bienestar, pero que encuentran en las excusas el principal obstáculo.

La aplicación brinda apoyo constante y permite construir rutinas de ejercicio y planes de alimentación completamente personalizados, adaptados a las limitaciones, objetivos y estilo de vida de cada usuario. Todo esto es posible gracias a la integración de Inteligencia Artificial con un backend propio que valida y ordena la información, generando de esta manera planes a medida del usuario de forma automática.

Además, FitPocket está diseñada para fomentar la constancia mediante una interacción diaria entre el usuario y la aplicación: registrar el cumplimiento del día, ver la racha de días consecutivos y visualizar el progreso a través de un gráfico de adherencia semanal son parte de la experiencia cotidiana. La vista mensual del progreso está planificada y aún no fue desarrollada.

La experiencia diaria con FitPocket

Una vez que el plan queda guardado en la cuenta, comienza la convivencia con FitPocket. Cada día, el usuario cuenta con un checklist para registrar su avance tanto en la rutina de ejercicio como en la dieta, lo que actualiza automáticamente el porcentaje de adherencia, la racha y el gráfico de progreso.

Hoy, cuando la adherencia de la semana cae por debajo del 50% y ya hay al menos tres días contados, el inicio muestra un aviso. La notificación por mail o push prevista originalmente (72 horas sin alcanzar el 60% de los objetivos) sigue planificada y todavía no está implementada.

La intención de que el usuario pueda ajustar su plan en cualquier momento, sin esperar alertas, se mantiene. En la versión actual puede reemplazar cualquier ejercicio por una de sus alternativas desde el detalle del plan; volver a generar el plan completo desde la interfaz está pendiente. La filosofía detrás de esto no es generar culpa ni presión, sino ofrecer apoyo constante y flexibilidad real para que cada persona pueda seguir avanzando hacia sus metas a su propio ritmo.

Comunidad y progreso compartido

FitPocket contempla la posibilidad de exportar el historial de progreso para compartirlo con amigos o en redes sociales, fomentando la creación de una comunidad que se apoye y motive mutuamente. También contempla un espacio dedicado para que los usuarios puedan enviar sugerencias, críticas constructivas e ideas de mejora, participando activamente en el desarrollo futuro de la aplicación. Ambas funcionalidades están planificadas y aún no fueron desarrolladas.

El equipo detrás de FitPocket

FitPocket es desarrollada por un equipo de cuatro programadores que buscan complementarse en conocimiento, creatividad y distribución estratégica de tareas. Cada integrante tiene la libertad de trabajar con las herramientas con las que se siente más cómodo, centralizando los avances en un documento compartido para mantenerse al día (https://docs.google.com/document/d/1_0-o0bnqBjfOTD8nyZUrwW2c50bflHqgvybm0cUQm_o/edit?usp=sharing), establecer metas de trabajo diarias y apoyarse mutuamente ante cualquier duda o bloqueo.

El debate es nuestra principal herramienta para resolver problemas que parecen no tener salida, y la lluvia de ideas nos ayuda a encontrar el camino más simple y eficiente. Lo que no se sabe, se investiga y se comparte con el resto del equipo para aprender en conjunto.

El código se versiona en GitHub. El trabajo se integra en la rama development y main se reserva para la entrega.

Descripción de Usuarios de Sistema

FitPocket contempla dos tipos de usuarios con diferentes niveles de acceso e interacción con el sistema. El backend define además un rol de administrador, que hoy solo se usa a través de la API.

Usuario Invitado

Es cualquier persona que accede a la aplicación sin haber creado una cuenta. No requiere autenticación y su sesión no persiste entre visitas. Tiene acceso a las funcionalidades públicas del sistema.

Atributo	Detalle
Autenticación	No requerida
Acceso	Página de inicio, formulario de registro, inicio de sesión (modal y página) y recuperación de contraseña. La calculadora de IMC para invitados prevista originalmente no está en la página de inicio actual.
Restricciones	No puede ver ni generar planes, no tiene checklist, no accede a gráficas ni perfil. Si intenta entrar a una pantalla privada, la aplicación lo envía al inicio de sesión.
Conversión	Puede registrarse para convertirse en Usuario Registrado

Usuario Registrado

Es una persona que completó el proceso de registro y el onboarding. Tiene una cuenta activa con datos de perfil, un plan generado por IA y acceso a las funcionalidades del sistema.

Atributo	Detalle
Autenticación	Requerida (email + contraseña, token JWT de 7 días)
Acceso	Todas las funcionalidades disponibles del sistema
Datos asociados	Perfil físico, plan activo (rutina + dieta), planes anteriores archivados, checkins diarios y adherencia semanal. Las alertas guardadas en base de datos previstas originalmente no se implementaron; el aviso de adherencia baja se calcula al abrir el inicio.
Capacidades	Ver su plan día por día, reemplazar ejercicios por alternativas, completar el checklist diario, ver su racha y su adherencia semanal, recuperar su contraseña y usar la aplicación en español o inglés. Pendientes: modificar el plan completo, registrar peso, exportar progreso y enviar sugerencias.

Usuario Administrador

Rol definido en el backend (campo rol del usuario). Permite listar todos los usuarios y verificar la integridad de los datos mediante dígitos verificadores. No tiene pantallas propias: se usa a través de la API.

Funcionalidades y Tecnicismos

Para que se puedan cumplir todos los objetivos del programa, desarrollamos algunas funciones que van a estar estructuradas de una forma completa y dedicada. En esta sección se explican más a fondo de forma lógica, indicando qué está construido y qué queda pendiente.

Información del Usuario

Cuando el usuario se registra en el sistema ingresa nombre, email y contraseña (mínimo 8 caracteres). Luego tiene que ingresar obligatoriamente su información personal para la creación de su plan de entrenamiento y dieta, la cual se solicita mediante un onboarding de **cuatro**

pasos mostrados en la misma interfaz. Si vuelve a un paso anterior, la aplicación conserva lo que ya había cargado. La información que se pide es la siguiente:

- **Peso**, en kilogramos (entre 30 y 300). El ingreso en libras previsto originalmente no está disponible.
- **Altura**, en centímetros (entre 100 y 250). El ingreso en pies previsto originalmente no está disponible.
- **Edad**. El documento original fijaba un mínimo de 15 años; el formulario actual acepta de 10 a 99.
- **Sexo** (femenino o masculino).
- **Objetivo**, de elección única entre:
 - Perder grasa
 - Ganar músculo
 - Resistencia
 - Salud general
 - Mejorar dieta/bienestar (el modelo de datos lo admite, pero el formulario actual no lo ofrece)
- **Nivel de experiencia** en las rutinas de ejercicio: principiante, intermedio o avanzado.
- **Días disponibles** de la semana y **minutos por sesión** (30, 45, 60 o 90).
- **Tipo de alimentación**: sin restricción, vegetariana, vegana o sin gluten. El modelo de datos admite también keto.
- **Presupuesto mensual** en pesos argentinos, opcional. Si se carga, tiene que ser mayor a cero.
- **Limitaciones físicas**. Si el usuario indica alguna, el plan se adapta a ella. Las opciones actuales son:
 - Dolor lumbar
 - Rodilla operada
 - Hombro
 - Asma
 - Hipertensión
 - Ninguna
 - Un campo de texto libre para que el usuario aclare lo que necesite

Las categorías previstas originalmente (alergias, preferencias, problemas motrices, lesiones y discapacidades) se cubren hoy con el tipo de alimentación, las limitaciones físicas y el campo de aclaración. Las limitaciones y la aclaración son datos de salud y se guardan encriptadas.

Con todos estos datos ingresados, el backend le pide a una IA el plan semanal, que contiene la rutina, la dieta y la meta diaria estructuradas en un JSON, y lo adapta a la interfaz en el frontend. Antes de guardarlo, el backend revisa la respuesta: conserva solo los días que el usuario eligió, reemplaza o quita ejercicios contraindicados para sus limitaciones y registra el presupuesto y el nivel que el usuario indicó. El plan se genera en el idioma en que el usuario tiene la interfaz.

El IMC se calcula en tiempo real en el primer paso del onboarding y se muestra en el inicio del usuario registrado. El cálculo para usuarios invitados previsto originalmente no está disponible en la versión actual.

Cuando un usuario ya tiene una cuenta y necesita ingresar de nuevo, inicia sesión con el email de la cuenta y la contraseña. El ingreso con nombre de usuario previsto originalmente no está disponible. Si olvidó la contraseña, puede pedir un enlace por mail para elegir una nueva; el enlace vence a los 30 minutos y se usa una sola vez.

Plan de entrenamiento y dieta

El usuario ve su plan completo día por día: los ejercicios con series, repeticiones y descanso, y las cuatro comidas con sus calorías y el costo estimado del día. Cada ejercicio puede reemplazarse por una de sus alternativas; ese reemplazo no se conserva al recargar la página. Cuando se genera un plan nuevo, el anterior queda archivado.

Registro de Avance

Cada día, el usuario tiene acceso a un checklist para registrar su avance en su rutina y en su dieta:

- Marcar como completado, parcial o no realizado, por separado para la rutina y para la dieta. **Implementado.**
- Agregar un registro de peso diario de manera opcional. **Pendiente:** hoy se muestra el peso inicial cargado en el perfil.
- Calcular la adherencia en porcentaje. **Parcial:** se calcula la semanal (últimos 7 días); la mensual está pendiente.
- Gráficas de progreso de peso (línea) y adherencia (barras). **Parcial:** existe el gráfico de barras de adherencia por día; el de peso está pendiente.

La adherencia semanal se calcula así: la rutina cuenta solo en los días en que el plan tiene entrenamiento, la dieta cuenta todos los días, y no se cuentan los días anteriores al primer plan del usuario. Las fechas se toman en horario de Argentina. El inicio muestra además la racha de días consecutivos con registro.

De esta forma cada usuario puede manejar de forma ordenada su rutina diaria. La meta original de cumplir el 60% de los objetivos y el envío de un mail y una notificación si no se alcanza en 72 horas siguen planificados; hoy la aplicación muestra un aviso en el inicio cuando la adherencia semanal baja del 50%.

El objetivo de que el sistema permita siempre las modificaciones que quiera hacer el usuario, en la dieta o en su plan de entrenamiento, se mantiene; hoy está cubierto de forma parcial por el reemplazo de ejercicios.

Vista de su Perfil

FitPocket contempla que el usuario vea la información de su perfil en un apartado especial, para modificar ciertos datos o verificarlos, con todo lo ingresado en el onboarding y la gráfica del

IMC. En la versión actual la pantalla de perfil no está desarrollada: el backend ya permite consultar, actualizar y eliminar la cuenta, y el inicio muestra el nombre, el nivel y el IMC del usuario.

Comunidad

Compartir Avances Propios

Se contempla que cada usuario pueda exportar su historial de progreso en formato PNG para compartirlo con amigos o en redes sociales, con el objetivo de fomentar la creación de una comunidad. Pendiente de desarrollo.

Ideas para Comunidad

Se contempla una sección dedicada a los usuarios para que puedan enviar sugerencias, críticas constructivas e ideas de mejora, para que la aplicación tenga un gran desarrollo en el futuro. Pendiente de desarrollo.

Idiomas

La interfaz está disponible en español e inglés, con un selector visible en todas las pantallas. La elección se recuerda entre visitas.

Requisitos Funcionales y No Funcionales

Se conservan todos los requisitos originales (RF01 a RF15 y RNF01 a RNF10). Los que se agregaron en esta versión continúan la numeración.

Funcionales

ID	Descripción	Estado
RF01	El sistema debe permitir que un usuario invitado se registre ingresando nombre, email y contraseña.	Implementado. Valida nombre, formato de email y contraseña de 8 caracteres o más.
RF02	El sistema debe permitir que un usuario registrado inicie sesión con email y contraseña.	Implementado
RF03	El sistema debe guiar al usuario a través de un onboarding de cuatro pasos (antes: dos) para recopilar datos físicos, objetivos, disponibilidad y limitaciones.	Implementado
RF04	El sistema debe calcular el IMC en tiempo real tanto para usuarios invitados como para usuarios registrados.	Parcial. Se calcula en el onboarding y en el inicio del usuario registrado; no hay calculadora para invitados.
RF05	El sistema debe generar automáticamente un plan semanal de rutina y dieta personalizado mediante IA, a partir de los datos del onboarding.	Implementado

ID	Descripción	Estado
RF06	El sistema debe crear un checklist diario para que el usuario registre el cumplimiento de su rutina y dieta (completado, parcial o no realizado).	Implementado
RF07	El sistema debe permitir al usuario registrar su peso diario de forma opcional al completar el checklist.	Pendiente
RF08	El sistema debe calcular y mostrar el porcentaje de adherencia semanal y mensual del usuario.	Parcial. Semanal implementada; mensual pendiente.
RF09	El sistema debe mostrar gráficas de progreso: evolución del peso (línea) y adherencia (barras).	Parcial. Barras de adherencia implementadas; gráfico de peso pendiente.
RF10	El sistema debe enviar una notificación por email y/o push si el usuario no completa al menos el 60% de sus objetivos en 72 horas.	Pendiente. Hoy se muestra un aviso en el inicio cuando la adherencia semanal baja del 50%.
RF11	El sistema debe regenerar automáticamente el plan del usuario cuando su adherencia cae por debajo del 50% durante 7 días.	Pendiente
RF12	El sistema debe permitir al usuario modificar su plan en cualquier momento, sin necesidad de esperar una alerta.	Parcial. Puede reemplazar ejercicios por alternativas; regenerar el plan desde la interfaz está pendiente.
RF13	El sistema debe mostrar al usuario su información de perfil y permitirle editar sus datos físicos y preferencias.	Parcial. El backend lo permite; la pantalla de perfil está pendiente.
RF14	El sistema debe permitir al usuario exportar su historial de progreso en formato PNG para compartirlo.	Pendiente
RF15	El sistema debe ofrecer un espacio donde los usuarios puedan enviar sugerencias e ideas de mejora.	Pendiente
RF16	El sistema debe permitir recuperar la contraseña mediante un enlace enviado por mail, válido por 30 minutos y de un solo uso. (Nuevo)	Implementado
RF17	El sistema debe ofrecer la interfaz en español e inglés y generar el plan en el idioma elegido. (Nuevo)	Implementado
RF18	El sistema debe mostrar el detalle del plan día por día y permitir reemplazar un ejercicio por una de sus alternativas. (Nuevo)	Implementado
RF19	El sistema debe adaptar el plan a las limitaciones físicas del usuario, evitando ejercicios contraindicados. (Nuevo)	Implementado. Reglas por limitación en el pedido a la IA y un filtro posterior por nombre de ejercicio.
RF20	El sistema debe mostrar la racha de días consecutivos con registro. (Nuevo)	Implementado
RF21	El sistema debe permitir al usuario eliminar su cuenta junto con sus planes y registros. (Nuevo)	Parcial. Disponible en el backend; falta el botón en la interfaz.
RF22	El sistema debe permitir a un administrador listar usuarios y verificar la integridad de los datos. (Nuevo)	Parcial. Disponible en el backend; sin pantallas.

Requisitos no funcionales

ID	Descripción	Estado
RNF01	El sistema debe cifrar las contraseñas de los usuarios usando el algoritmo bcrypt antes de almacenarlas. Nunca se guarda texto plano.	Implementado
RNF02	El sistema debe autenticar a los usuarios mediante tokens JWT con una vigencia de 7 días.	Implementado. Además, al cambiar la contraseña dejan de valer los tokens anteriores.
RNF03	Las rutas protegidas del backend deben validar el token JWT antes de procesar cualquier solicitud.	Implementado. También se verifica que la cuenta siga existiendo.
RNF04	El sistema debe garantizar que no existan dos usuarios con el mismo email mediante una restricción de unicidad en la base de datos.	Implementado
RNF05	El sistema debe garantizar que no existan dos checkins del mismo usuario para la misma fecha, mediante un índice compuesto único.	Implementado
RNF06	El sistema debe responder a las solicitudes del frontend en un tiempo razonable, optimizando las consultas a la base de datos.	Implementado. La generación del plan depende de la IA y tarda entre 5 y 60 segundos.
RNF07	La interfaz de usuario debe ser responsiva y adaptarse a dispositivos móviles y de escritorio.	Implementado. Verificado a 390 px y 1440 px.
RNF08	El sistema debe ser desplegable como PWA, permitiendo su instalación en dispositivos móviles sin pasar por una tienda de aplicaciones.	Pendiente
RNF09	El sistema debe mantener un historial completo de versiones de planes generados por usuario, sin sobrescribir versiones anteriores.	Parcial. Los planes anteriores se conservan como archivados; el número de versión todavía no se incrementa.
RNF10	El código del sistema debe estar organizado siguiendo una arquitectura MVC, separando modelos, controladores y rutas.	Implementado. Se suman capas de servicios, middleware y utilidades.
RNF11	Los datos sensibles del perfil (nombre, limitaciones y aclaración) deben almacenarse encriptados. (Nuevo)	Implementado (AES-256-CBC)
RNF12	El sistema debe registrar una bitácora con las acciones realizadas sobre la API. (Nuevo)	Implementado
RNF13	El sistema debe usar dígitos verificadores para detectar alteraciones en los datos de los usuarios. (Nuevo)	Implementado
RNF14	Las pantallas privadas del frontend deben exigir sesión y redirigir al inicio de sesión cuando no la hay o venció. (Nuevo)	Implementado
RNF15	La interfaz debe cumplir pautas básicas de accesibilidad: contraste de texto de 4,5:1, áreas táctiles de 44 px y foco de teclado visible. (Nuevo)	Implementado
RNF16	El sistema debe tolerar los límites de uso del proveedor de IA, reintentando en lugar de mostrar un error.	Implementado. Reintenta hasta 90 segundos.

ID	Descripción	Estado
	(Nuevo)	
RNF17	Las fechas de los registros diarios deben tomarse en horario de Argentina. (Nuevo)	Implementado

Tecnologías y Herramientas

La arquitectura cambió respecto de la planificación inicial. En la primera versión de este documento toda la lógica de negocio vivía en n8n, la persistencia se resolvía con archivos JSON locales y se preveían varias APIs externas. Durante el desarrollo el equipo optó por un backend propio y una base de datos en la nube. A continuación se describe lo que el proyecto usa hoy.

Frontend

El frontend está desarrollado con **React 19 + Vite 8**, usando JavaScript puro. La navegación entre secciones se maneja con **React Router 7**. El estado se resuelve con los hooks de React y la sesión se guarda en el navegador; no se usan Zustand ni React Query, previstos originalmente. Para los estilos se usa **Tailwind CSS v4** junto con hojas CSS propias basadas en variables de diseño, y componentes de shadcn/ui. Los gráficos de progreso están hechos con CSS propio, sin Recharts. Los textos se traducen con **i18next** (español e inglés) y las tipografías son Space Grotesk y DM Sans. La instalación como PWA sigue planificada.

Backend

La lógica de negocio vive en un backend propio hecho con **Node.js + Express 5**, organizado en rutas, controladores, modelos, servicios, middleware y utilidades. Desde ahí se registran y autentican los usuarios, se guarda el perfil, se pide y valida el plan, se registran los checkins, se calcula la adherencia y se envían los mails. El frontend se comunica con el backend mediante una API REST autenticada con tokens JWT. n8n se evaluó en la etapa inicial y ya no forma parte del proyecto.

Base de datos

La persistencia se resuelve con **MongoDB Atlas**, accedida mediante **Mongoose**. Las colecciones son usuarios, planes, checkins y bitácora. La migración desde archivos JSON locales, que el documento original recomendaba para una versión a mayor escala, ya está hecha. El proyecto incluye scripts para hacer y restaurar copias de respaldo de la base.

Inteligencia Artificial y servicios externos

El plan se genera con la API de **Groq** (modelo openai/gpt-oss-120b), en dos pedidos: uno para la rutina y otro para la dieta. Los mails de recuperación de contraseña se envían por **SMTP** con

Nodemailer. Las APIs previstas originalmente (ExerciseDB, Nutritionix, OpenAI GPT-4o-mini, Web Push con VAPID y SendGrid) no se integraron. Todas las claves se guardan en variables de entorno del backend, sin exposición al frontend.

Seguridad

Las contraseñas se guardan con bcrypt, las sesiones usan JWT de 7 días, los datos sensibles del perfil se encriptan con AES-256-CBC, cada usuario lleva un dígito verificador y las acciones sobre la API quedan registradas en una bitácora.

Despliegue

El repositorio incluye un Dockerfile para el backend y otro para el frontend. El despliegue previsto originalmente (frontend en Vercel o Netlify) sigue siendo una opción válida; a la fecha de este documento no hay un despliegue en la nube documentado y la aplicación se ejecuta en entorno local.

Capa	Planificado originalmente	Actual
Frontend	React 18, React Router v6, Zustand, React Query, Recharts	React 19, React Router 7, hooks de React, gráficos propios, i18next
Estilos	Tailwind CSS v4 + shadcn/ui	Tailwind CSS v4 + shadcn/ui + CSS propio con variables
Lógica de negocio	n8n con webhooks	Backend propio en Node.js + Express 5
Persistencia	Archivos JSON locales	MongoDB Atlas con Mongoose
IA	OpenAI GPT-4o-mini	Groq (openai/gpt-oss-120b)
Mails	SendGrid	SMTP con Nodemailer
Notificaciones	Web Push con VAPID	Pendiente
Instalación	PWA	Pendiente

Diagramas

Diagrama de Casos de Uso

Se conserva el diagrama original. Dos aclaraciones para leerlo hoy: el actor "Sistema N8N / IA" corresponde al backend propio junto con la IA de Groq, y no todos los casos de uso están desarrollados. La tabla que sigue indica el estado de cada uno y agrega los casos nuevos.

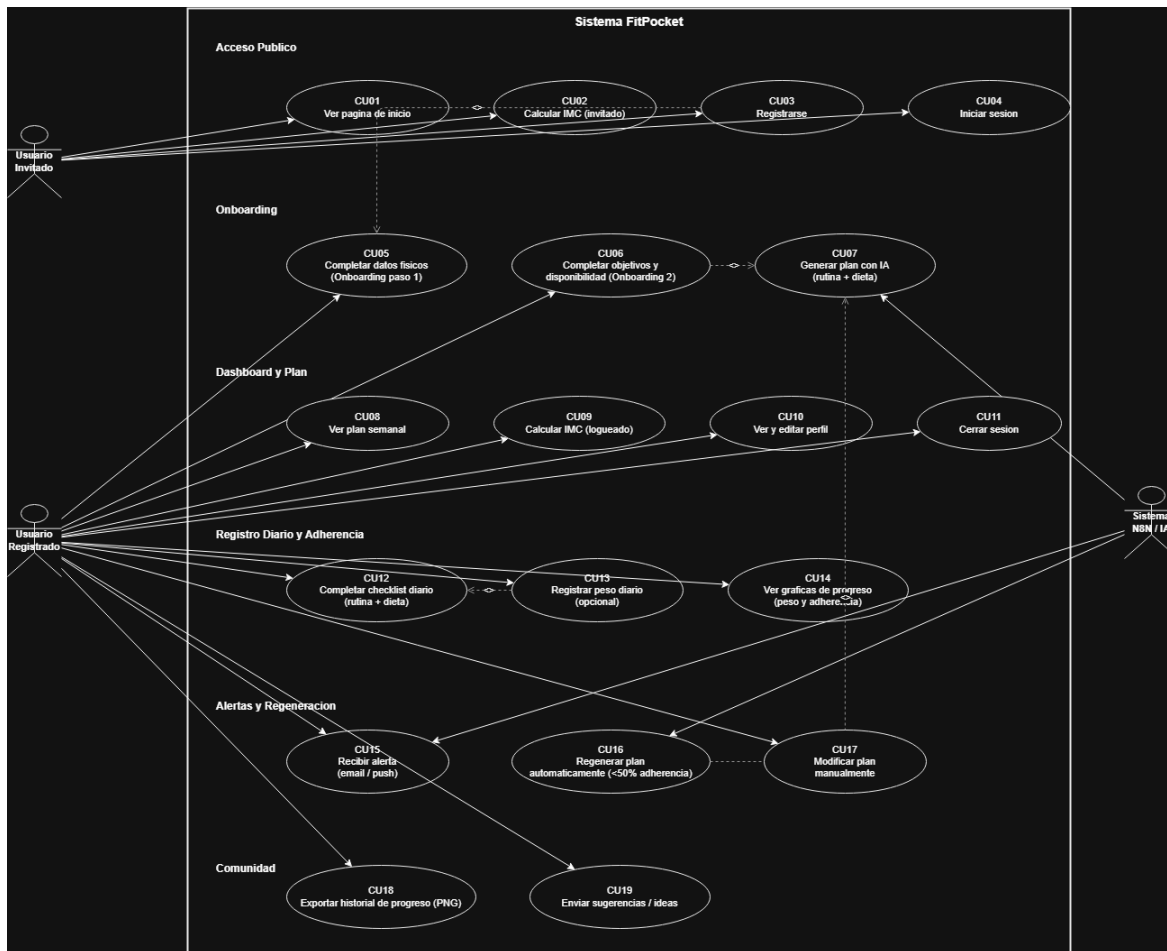


Diagrama de casos de uso (versión original).

Caso de uso	Descripción	Estado
CU01	Ver página de inicio	Implementado
CU02	Calcular IMC (invitado)	Pendiente
CU03	Registrarse	Implementado
CU04	Iniciar sesión	Implementado
CU05	Completar datos físicos (onboarding paso 1)	Implementado
CU06	Completar objetivos y disponibilidad (hoy pasos 2, 3 y 4)	Implementado
CU07	Generar plan con IA (rutina + dieta)	Implementado
CU08	Ver plan semanal	Implementado
CU09	Calcular IMC (logueado)	Implementado
CU10	Ver y editar perfil	Parcial. Solo en el backend.
CU11	Cerrar sesión	Implementado
CU12	Completar checklist diario (rutina + dieta)	Implementado
CU13	Registrar peso diario (opcional)	Pendiente
CU14	Ver gráficas de progreso (peso y adherencia)	Parcial. Solo adherencia.
CU15	Recibir alerta (email / push)	Pendiente. Hay un aviso en el

Caso de uso	Descripción	Estado
		inicio.
CU16	Regenerar plan automáticamente (<50% adherencia)	Pendiente
CU17	Modificar plan manualmente	Parcial. Reemplazo de ejercicios.
CU18	Exportar historial de progreso (PNG)	Pendiente
CU19	Enviar sugerencias / ideas	Pendiente
CU20	Recuperar contraseña por mail (nuevo)	Implementado
CU21	Reemplazar un ejercicio por una alternativa (nuevo)	Implementado
CU22	Cambiar el idioma de la interfaz (nuevo)	Implementado
CU23	Eliminar la cuenta (nuevo)	Parcial. Solo en el backend.

Diagrama de Clase

El diagrama se actualizó para reflejar los modelos actuales. Respecto de la versión anterior: la clase Alerta no se implementó, se agregó Bitacora, el usuario suma rol, dígito verificador y los campos de recuperación de contraseña, y el checkin guarda el estado y el puntaje de rutina y dieta en lugar del peso y un porcentaje.

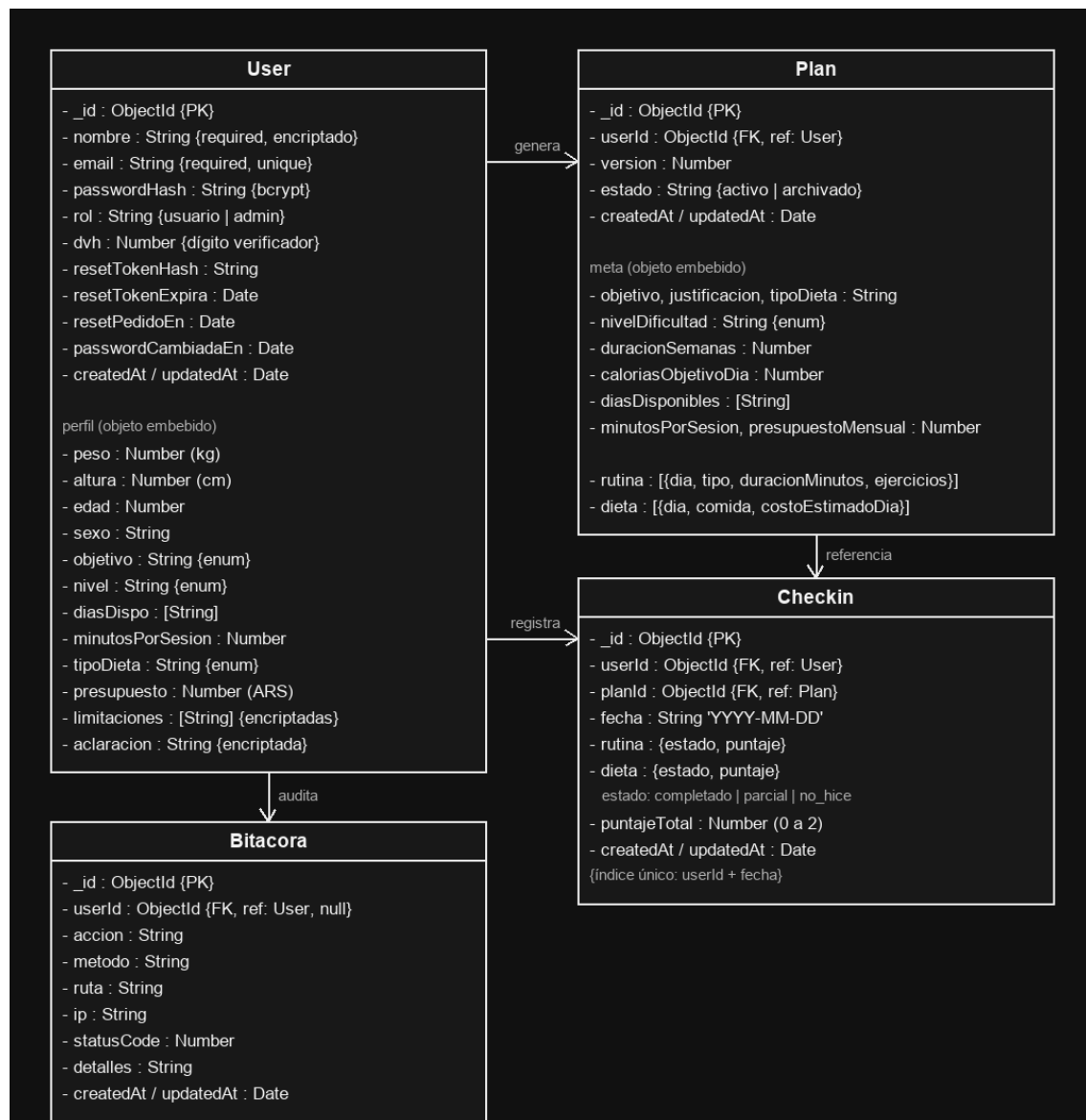


Diagrama de clases (actualizado).

Escalabilidad

FitPocket fue diseñado como un MVP funcional con una arquitectura que permite crecer de forma ordenada. A continuación se describen las funcionalidades que podrían incorporarse en versiones futuras del sistema. Se mantienen todos los puntos de la versión anterior; los que ya se concretaron están indicados.

Infraestructura y base de datos

- Migración de archivos JSON locales a MongoDB Atlas. **Ya realizado.** Quedan como mejora las réplicas para mayor disponibilidad y tolerancia a fallos.

- Implementación de particionado de colecciones por usuario para optimizar el rendimiento con grandes volúmenes de datos.
- Incorporación de estrategias de backup automático y restauración programada. **Parcial:** existen scripts manuales de respaldo y restauración.
- Uso de caché (por ejemplo, Redis) para reducir consultas repetidas a la base de datos.
- Plan pago del proveedor de IA para atender muchos usuarios generando planes al mismo tiempo. (Nuevo)

Funcionalidades de Usuario

- Incorporación de un rol de Administrador con acceso a métricas globales, gestión de usuarios y moderación de sugerencias. **Parcial:** el rol existe en el backend, sin pantallas.
- Planes de suscripción premium que habiliten funcionalidades avanzadas como planes de mayor duración, análisis nutricional detallado o integración con wearables.
- Sistema de comunidad con perfiles públicos, seguimiento entre usuarios y comparativa de progreso.
- Soporte para múltiples idiomas (internacionalización) para alcanzar usuarios de habla no hispana. **Ya realizado** para español e inglés.
- Pantalla de perfil, registro de peso, adherencia mensual, exportación del progreso y sección de sugerencias, que figuran como requisitos pendientes. (Nuevo)

Integraciones Externas

- Integración con APIs de dispositivos y wearables (Google Fit, Apple Health) para sincronizar datos de actividad física automáticamente.
- Incorporación de un módulo de videollamadas o coaching con profesionales de la salud y el deporte.
- Publicación de una API pública documentada para que terceros puedan integrar los planes generados por FitPocket en otras aplicaciones.
- Notificaciones por mail y push, y regeneración automática del plan por baja adherencia. (Nuevo; corresponden a RF10 y RF11)

Seguridad y Control

- Implementación de autenticación de dos factores (2FA).
- Registro de bitácora técnica con trazabilidad de acciones por usuario. **Ya realizado.**
- Implementación de dígitos verificadores para garantizar la integridad de datos críticos en la base de datos. **Ya realizado.**
- Pruebas automatizadas del backend y del frontend. (Nuevo)